# Project Zomboid Modding: A Complete Deep Dive

## Overview

Project Zomboid's modding system is one of the most open and powerful in the survival game genre. The game's engine is split between **Java** (core engine, not moddable) and **Lua** (game logic, fully moddable), with a proprietary **Script language** (`.txt` files) used for defining items, recipes, and vehicles[^1]. Together these layers allow modders to create anything from a single new item to total conversion overhauls. The Steam Workshop hosts approximately **50,000+ modifications**[^2], making it one of the largest modding communities on the platform.

Project Zomboid's modding policy is permissive: The Indie Stone allows modders to edit virtually any part of the game, share mods freely, and even charge for them under certain conditions — as long as mods don't circumvent the game's purchase requirement[^3].

## Tools You'll Need

Before writing a single line of code, get these tools installed:

| Tool | Purpose | Where to Get |
|---|---|---|
| Visual Studio Code | Primary code editor — the community standard for PZ modding[^2] | https://code.visualstudio.com |
| Umbrella (VSCode ext) | Syntax highlighting for the Lua API[^2] | VSCode Extensions Marketplace |
| Project Zomboid Script Support (VSCode ext) | Syntax highlighting for ZedScript (item/recipe `.txt` files)[^2] | VSCode Extensions Marketplace |
| Zed Script (VSCode ext) | Additional script syntax support[^2] | VSCode Extensions Marketplace |
| GIMP or Paint.NET | Creating and editing item/poster textures[^4] | https://www.gimp.org |
| TileZed / WorldEd / BuildingEd | Official map editing tools (for map mods)[^5] | Bundled with PZ via Steam Tools |
| Audacity | Editing custom sounds (`.ogg` / `.wav` format)[^6] | https://www.audacityteam.org |

The **in-game debugger** is also essential. It allows you to test scripts live, examine game objects, spawn items, and catch errors without restarting[^7].

## Understanding the Three Modding Layers

### Layer 1 — ZedScript (`.txt` files)

The simplest entry point. Used to define **items, recipes, vehicles, and sounds**. No coding knowledge required — it's a property-value format[^1]. These files live in `media/scripts/`. If you want to add a new knife, canned food, car, or craftable item, this is your starting point.

### Layer 2 — Lua (`.lua` files)

The real power layer. Lua handles **game logic, UI, events, multiplayer synchronization, and custom mechanics**. PZ uses a scripted version of Lua (via the Kahlua VM on top of the Java engine)[^8]. Lua files are split into three contexts: `client`, `server`, and `shared`. If you want custom behaviors, UI windows, stat modifications, event triggers, or any dynamic gameplay, you work here[^9].

### Layer 3 — Java (source code)

The core engine. Modifying Java is **not recommended or officially supported**, though it's technically possible. The developers have committed to keeping everything moddable from Lua without requiring Java edits[1][8].

## The Mod Folder Structure

### Build 41 Structure (Classic)

The Build 41 folder layout is simple and flat:

```
YourMod/
├── mod.info
├── poster.png          (512x512 px preview image)
└── media/
    ├── scripts/        (.txt item/recipe definitions)
    ├── textures/       (.png icons and textures)
    ├── sound/          (.ogg or .wav audio files)
    ├── models/         (3D models for items/vehicles)
    └── lua/
        ├── shared/     (loads first; used by both client & server)
        ├── client/     (UI, context menus, client-side events)
        └── server/     (spawning logic, server events, farming)
```

Testing location: `C:\Users\YourUsername\Zomboid\mods\YourMod\`[^10]

### Build 42 Structure (New Versioned System)

Build 42 introduced a **version-layered** folder structure so a single mod can support multiple builds simultaneously[11][12]:

```
YourMod/
├── Contents/
│   └── mods/
│       └── YourMod/
│           ├── common/        (Shared resources for ALL versions)
│           │   ├── mod.info
│           │   └── media/
│           │       └── lua/
│           │           ├── client/
│           │           ├── server/
│           │           └── shared/
│           ├── 42/            (Build 42-specific files)
│           │   ├── mod.info
│           │   └── poster.png
│           └── 41/            (Build 41 files, optional)
│               ├── mod.info
│               └── poster.png
└── workshop.txt
```

The `common/` folder holds code that runs on any build. The `42/` and `41/` folders hold build-specific metadata and overrides[^12]. If you're building for B42 only, put your `media/` content inside `common/` and create the `42/` folder with just the `mod.info`[^11].

Testing location on Windows: `C:\Users\YourUsername\Zomboid\Workshop\`[^12]

### The `mod.info` File

Every mod must have a `mod.info` file. In Build 42 this lives inside the versioned folder (`42/` or `common/`)[^6]:

```
name=My Survival Mod
id=MySurvivalMod
description=Adds new survival items and mechanics.
poster=poster.png
url=https://github.com/yourrepo
```

Key fields:

- `id` — Unique mod identifier used internally and in server configs. No spaces
- `name` — Display name in the in-game mod menu
- `poster` — Path to your 512x512 preview image[^4]
- `url` — Optional link (forum post, GitHub, etc.)

# Writing ZedScript: Items, Recipes, and More

## Module Structure

All script objects must be wrapped inside a `module` block. Using `Base` as your module name shares namespace with vanilla items (meaning you can overwrite them), while a custom module name prevents conflicts[^13]:

```
/** This is a comment — use these liberally **/
module MyMod {
    imports {
        Base   /** lets you reference Base.Nails as just Nails **/
    }

    item MyKnife
    {
        Type           = Weapon,
        DisplayName     = My Custom Knife,
        Icon           = Item_MyKnife,
        Weight         = 0.5,
        MaxRange       = 0.9,
        MinAngle       = 0.0,
        SwingTime      = 2,
        MinimumSwingTime = 2,
        DoorDamage      = 1,
        MaxHitCount    = 1,
        HitAngleMod     = -30,
        CriticalChance  = 20,
        CritDmgMultiplier = 3,
    }
}
```

## Item Types

| Type | Description | Key Properties |
|---|---|---|
| `Normal` | Basic non-consumable item | `Weight`, `Icon` |
| `Food` | Consumable; affects hunger/thirst | `HungerChange`, `ThirstChange`, `DaysFresh` |
| `Drainable` | Has charges/uses before depletion | `UseDelta` (0.1 = 10 uses) |
| `Weapon` | Equippable weapon | `MaxRange`, `SwingTime`, `MaxDamage` |
| `Container` | Stores other items | `Capacity`, `CanBeEquipped` |
| `Clothing` | Worn by character | `BodyLocation`, `Temperature`, `Insulation` |
| `Literature` | Readable book/magazine | `CanBeRead`, `LvlSkillTrained` |

### Recipe Syntax

Recipes use `:` instead of `=` for property assignment[^13]:

```
recipe Craft Makeshift Spear
{
    /** Ingredients: **/
    keep WoodenPlank,           /** 'keep' = not consumed **/
    NailsBox/ButterKnife,       /** '/' = OR (either works) **/

    Result:WoodenSpear=1,
    Sound:WoodConstruction,
    Time:50.0,
    OnCreate:MakeSpear_OnCreate,     /** calls a Lua function **/
    OnGiveXP:Give5CarpentryXP,
    Category:Survival,
    NeedToBeLearn:true,
}
```

### Sound Block

Sounds must be declared in a `sound` block to be heard beyond 20 tiles in multiplayer[^13]:

```
sound MyWeaponSwing
{
    category = Item,
    clip
    {
        file = media/sound/myswing.ogg,
        distanceMax = 40,
    }
}
```

## Writing Lua: Events, Logic, and Behavior

### The Events System

Events are the backbone of PZ mod logic. The game fires events on specific actions, and your Lua code **hooks into** those events to run custom code[^14]:

```
-- Basic event hook pattern:
local function myFunction(parameters)
    -- your logic here
end
Events.EventName.Add(myFunction)
```

## Essential Events Reference

| Event | When It Fires | Parameters |
|---|---|---|
| `OnGameStart` | When a save is loaded | none |
| `OnPlayerUpdate` | Every player update tick | `player` |
| `OnZombieUpdate` | Every zombie update tick | `zombie` |
| `EveryTenMinutes` | Every 10 in-game minutes | none |
| `OnKeyPressed` | Any key press | `key` (int code) |
| `OnKeyStartPressed` | Key held down (continuous) | `key` |
| `OnWeaponHitCharacter` | Weapon connects with a target | `attacker, target, weapon, dmg` |
| `OnWeaponSwing` | Weapon swing initiates | `chr, weapon` |
| `OnCreateLivingCharacter` | New character spawns | `player, desc` |
| `OnFillInventoryObjectContextMenu` | Right-click inventory menu opens | `player, context, items` |
| `OnFillWorldObjectContextMenu` | Right-click world object menu | `player, context, worldobjects` |
| `OnSave` | Game is saved | none |
| `OnLoad` | Game loads | none |
| `AddXP` | XP is added to a skill | `player, perk, xp` |
| `LevelPerk` | A skill levels up | `player, perk, level, skillbook` |

The full event list can be dumped in-game using the **Dump Events** mod from Steam Workshop[^14].

## Lua File Context Rules

The context (folder) your Lua file lives in determines when and where it runs[^6]:

- `media/lua/shared/` — Loads first; runs on both client and server. Use for data tables, utility functions, and anything both sides need to share
- `media/lua/client/` — Runs only on each player's game client. Use for UI, rendering, input handling, and client-side events
- `media/lua/server/` — Runs only on the dedicated server or the host. Use for item spawning, world events, and persistence logic

### Hello World Mod (Full Example)

This is a complete, working mod that makes the player say "I'm alive!" every 10 in-game minutes:

`mod.info`:

```
name=Hello Survivor
id=HelloSurvivor
description=A basic demonstration mod.
poster=poster.png
```

`media/lua/client/HelloSurvivor.lua`:

```
local function everyTenMinutes()
    local player = getPlayer()
    if player then
        player:Say("I'm alive!")
    end
end

Events.EveryTenMinutes.Add(everyTenMinutes)
```

### Multiplayer-Safe Player

In singleplayer, `getPlayer()` returns the local player. In multiplayer this can cause bugs. The safe pattern for server-side code is[^7]:

```
local function onPlayerUpdate(player)
    -- 'player' is passed directly — always correct in multiplayer
    local stats = player:getStats()
    stats:setHunger(0.0)   -- example: zero out hunger
end
Events.OnPlayerUpdate.Add(onPlayerUpdate)
```

### Adding a Custom Right-Click Context Menu Option

```
local function addContextMenu(player, context, items)
    local item = items[^1]
    if item and item:getType() == "Matches" then
        context:addOption("Flick Matches", player, function(pl)
            pl:Say("*flick*")
        end)
    end
end
Events.OnFillInventoryObjectContextMenu.Add(addContextMenu)
```

### Accessing the Java API from Lua

PZ exposes its entire Java class hierarchy to Lua. Key classes and how to access them[^7]:

```
local player = getPlayer()
local stats   = player:getStats()     -- Stats object
local body    = player:getBodyDamage() -- BodyDamage object
local inv     = player:getInventory()  -- ItemContainer

-- Read a value
local hunger  = stats:getHunger()   -- 0.0 = full, 1.0 = starving

-- Set a value
stats:setThirst(0.5)
stats:setPanic(0.0)

-- Add item to inventory
inv:AddItem("Base.Axe")

-- Spawn item in world
local square = getCell():getGridSquare(x, y, z)
addItemToSquare("Base.Axe", square)
```

Full Java API documentation (community-maintained): `https://zomboid-javadoc.com`[^15]

### Item Loot Distribution (Spawn Tables)

Items don't spawn in the world unless you tell the game where to put them. Distribution is handled via Lua files in `media/lua/server/Items/`[^4]:

```
-- DistributionMyMod.lua
-- This runs server-side and registers your item in loot tables.

-- Add item to "KitchenCounters" distribution table
table.insert(SuburbsDistributions["all"]["kitchencounter"]["items"], {
    "MyMod.MyCustomKnife", 5,   -- item ID, weight (higher = more common)
})

-- Add to procedural distribution (containers matching a type)
ProceduralDistributions.list["PoliceStorageGuns"] = {
    rolls = 3,
    items = {
        "MyMod.MyCustomGun", 10,
    },
}
```

### Textures and Icons

Item icons must be `.png` format and named with the `Item_` prefix[^4]:

- File path: `media/textures/Item_MyKnife.png`
- Reference in script: `Icon = MyKnife` (the prefix is implied)
- Recommended size: **32x32 pixels** for inventory icons

The mod poster (`poster.png`) should be **512x512 pixels** and lives at the root of the mod folder (inside the `42/` folder for Build 42)[^4].

## Map Modding with TileZed

Map mods are the most complex mod type. The workflow requires three separate tools bundled with Project Zomboid[^5]:

1. **WorldEd** — Top-down world layout editor. Creates the overall map layout, defines zones (foraging, zombie density, roads), and stitches together regions
2. **TileZed** — Tile editor for placing individual tiles and building templates on the map
3. **BuildingEd** — Interior/exterior building editor. Used to design individual structures that get placed on the map

### Map Modding Workflow Overview

1. **Create a master BMP image** in Paintshop/GIMP/Paint.NET — black pixels become land, colored pixels define zones (roads, forests, etc.)[^5]
2. **Import into WorldEd** and define zones (foraging areas, zombie spawn regions, road paths)
3. **Design buildings in BuildingEd** — place walls, floors, furniture, and objects tile by tile
4. **Place buildings in TileZed** onto the world layout
5. **Compile the map** — WorldEd generates the `.lot` files the game reads
6. **Add spawn points** via `spawnpoints.lua` and `spawnregions.lua`[^16]
7. **Package as a mod** — place generated files into your mod's `media/maps/` folder

Custom tiles require packaging your tileset images into the mod and referencing them in TileZed's tile definitions[^17]. All custom tile images need to be loaded by the game engine separately from the editor — tiles visible in TileZed will be **invisible in-game** unless the tile definition is properly declared in the mod[^18].

### Sandbox Options (Custom Mod Settings)

Mods can expose configurable options to players via `sandbox-options.txt` inside your `media/` folder. This lets users tweak your mod's behavior from the Sandbox Settings menu without editing files[^19]:

```
VERSION = 1,
option MySetting
{
    type = boolean,
    default = true,
    page = MyMod,
    translation = MySetting,
}
```

Access sandbox values in Lua:

```
local myVal = SandboxVars.MySetting
```

## Build 42 Migration: What Changed in 42.13

Build 42.13 (December 2025) introduced a new **identifier and registry system** that breaks many Build 41 mods[^20]. Key changes for modders:

- The **versioned folder structure** (`common/`, `42/`, `41/`) is now required for Workshop mods targeting B42
- Mod IDs in server configs require a **backslash prefix** in Build 42: `Mods=\MyMod;` instead of `Mods=MyMod;`[^21]
- Item/entity registries were reworked — mods that directly hook into registration events will need updating[^20]
- Build 42's new crafting system has new script block types for production chains, kilns, and crafting stations not present in B41

The official migration guide is available on The Indie Stone forums[^20].

## Publishing to Steam Workshop

### Step 1: Set Up the Workshop Folder

Move your completed mod into the Workshop directory:

- Windows: `C:\Users\YourUsername\Zomboid\Workshop\YourMod\`[^12]

Your mod folder needs a `workshop.txt` file alongside `Contents/`[^12]:

```
version=1
id=0                  (0 until published; Steam assigns the real ID)
```

```
title=My Mod Name
description=What your mod does.
visibility=public      (public / friends / private)
tags=Build 42
```

### Step 2: Prepare Your Workshop Thumbnail

Create a preview image at **256x256 pixels** and save as `preview.png` in the root of the Workshop folder (next to `workshop.txt`)[^19]. This is separate from `poster.png` (shown in-game mod menu).

### Step 3: Upload via In-Game Tool

1. Launch Project Zomboid
2. Go to **Main Menu → Workshop → Create and Update Items**
3. Select your mod from the list
4. Click **Upload** — Steam handles the rest[^22]

After the first upload, Steam assigns a Workshop ID. Paste it into your `workshop.txt`'s `id=` field for future updates.

### Step 4: Update Your Workshop Page

On the Steam Workshop page:

- Add a detailed description (what the mod adds, configuration options, credits)
- Upload screenshots or gameplay images
- Tag accurately (Build 41/42, QoL, Weapons, Map, Overhaul, etc.)
- Respond to comments — active modders earn long-term subscribers

## Debugging Your Mod

- **Enable debug mode:** Launch PZ with `-debug` flag via Steam launch options[^7]
- **Console log:** Found at `C:\Users\YourUsername\Zomboid\console.txt` — all Lua errors and print statements appear here
- **In-game debug menu:** Press `F11` in debug mode; allows spawning items, teleporting, examining objects
- **Print to console from Lua:** `print("My value: " .. tostring(myVar))`
- **Test locally first:** Always test in singleplayer before uploading

Common errors:

- `attempt to index a nil value` — You're accessing a property on an object that doesn't exist yet. Check your event timing
- `no such field` — Typo in an item/recipe property name or wrong syntax (`:` vs `=`)

- Items not appearing in loot — Distribution table not loaded server-side, or wrong container key
- Mod not loading — `mod.info` missing, wrong folder path, or Build 42 versioning structure incorrect

## Mod Types: Complexity Tiers

| Mod Type | Skill Required | Core Tools | Time Estimate |
|---|---|---|---|
| New item(s) with custom icon | None / beginner | Text editor, GIMP | 1–2 hours |
| Custom recipe chain | Beginner | Text editor | 2–4 hours |
| Loot distribution change | Beginner | Text editor, basic Lua | 2–3 hours |
| New mechanic via Lua | Intermediate | VSCode, Lua knowledge | 1–3 days |
| Custom UI window | Intermediate–Advanced | VSCode, Java API docs | 2–5 days |
| Multiplayer-compatible system | Advanced | VSCode, Java API, MP testing | 1–2 weeks |
| New map region | Advanced | TileZed toolchain | Weeks–months |
| Total conversion overhaul | Expert | All tools | Months–years |

## Key Resources

- **PZ Wiki Modding Portal:** https://pzwiki.net/wiki/Modding — The most up-to-date official reference[^23]
- **The Indie Stone Forums (Modding):** https://theindiestone.com/forums — Official support, event lists, migration guides[^8]
- **FWolfe Modding Guide** (GitHub): https://github.com/FWolfe/Zomboid-Modding-Guide — Community reference for scripts and structure[^1]
- **PZ Java API Docs:** https://zomboid-javadoc.com — Full Java class reference for Lua access[^15]
- **LabX1 Build 42 Mod Template** (GitHub): https://github.com/LabX1/ProjectZomboid-Build42-ModTemplate — Ready-to-use B42 folder template[^12]
- **PZ Modding Discord:** https://pzwiki.net/wiki/PZ_Modding_Community — Active community help channel[^23]
- **Build 42.13 Migration Guide:** https://theindiestone.com/forums — Required reading if updating old mods[^20]

## References


1. Guide to modding various aspects of Project Zomboid - Guide to modding various aspects of Project Zomboid - FWolfe/Zomboid-Modding-Guide
2. How to create a mod [B42] - Project Zomboid - Steam Community - Video Guide. This is the best guides for b42 mod creation, it's a playlist of one famous modder - Si...
3. Modding Policy - Project Zomboid - Hello survivor! Here’s the modding policy for Project Zomboid. Long story short, you can make mods, ...
4. Modding Project Zomboid - Episode 3: Setting Up the Project - Episode 3: Setting Up the Project | Project Zomboid Modding Tutorial

Welcome back, survivors! In ...

5. Guide :: The One Stop TileZed Mapping Shop - Steam Community - Here you will find a (hopefully) comprehensive guide to map modding using TileZed, from scratch, to ...
6. Zomboid-Modding-Guide/structure/README.md at master · FWolfe/Zomboid-Modding-Guide - Guide to modding various aspects of Project Zomboid - FWolfe/Zomboid-Modding-Guide
7. [How To]: Modding Project Zomboid for Build 41 Multiplayer - Learn to write your own mods for Project Zomboid build 41! Make it multiplayer ready from the start!...
8. RoboMat's Modding Tutorials (Updated 12/11/2013) - To mod the game we somehow have to interact it. Lua's main purpose lies on extending existing progra...
9. Expanded Modding Support Details - Project Zomboid - Hello modding fans! In this blogpost we’ll detail an addition to the modding support in the next upd...
10. Guide :: Modding 101 : Add Items - This specifically targets adding custom items to game through your own mod....
11. Steam Community :: Guide :: How To Update Your Mod for Build 42 - Lot's of people are having problems figuring it out, it's pretty simple so here's how you do it....
12. GitHub - LabX1/ProjectZomboid-Build42-ModTemplate: A template and guide for creating Project Zomboid mods for Build 42, featuring a simple UI mod as a practical example. Includes folder structure explanations and cross-build compatibility guidance. - A template and guide for creating Project Zomboid mods for Build 42, featuring a simple UI mod as a ...
13. Zomboid-Modding-Guide/scripts/README.md at master - GitHub - Items, recipes, vehicles and similar stuff is defined in .txt files in the media/scripts/ directory....
14. Event list: How To and Current Event List - Hi, HOW TO Here is the method I used so you can easily do it yourself with any mod compilation and f...
15. API - Project Zomboid - This website and the documentation are not official! · Project Zomboid Javadoc 41.78 · Project Zombo...
16. How to make a map in Project Zomboid Build 41 - YouTube - ... Map, .tiles, & Lot Files 53:05 Spawnpoints.lua, spawnregions.lua & objects.lua 1:04:04 Making yo...
17. tutorial Custom Tilesets and fences for PZ Map Editor - I did a little tutorial to explain the process to adding your own tilesets to PZ and how to create y...

18. Project zomboid. custom tiles tutorial 01. getting custom ... - YouTube - Quick video on getting custom tiles in the project zomboid editors (buildinged, tiled and worlded) I...
19. Steam Community :: Discussions
20. Modding Migration Guide (42.13) - The Indie Stone Forums - A guide for mod authors on how to update your mods to the new identifier and registry system introdu...
21. Project Zomboid Build 42 Mods: Install And Fix Guide - Pine Hosting - Learn how to install Steam Workshop mods on Project Zomboid Build 42 server hosting. Fix mod mismatc...
22. How To: Upload a mod to Project Zomboid Workshop - How To: Upload a mod to Project Zomboid Workshop
23. PZ Modding Guides - Setting up a mod structure - This guide goes in detail on setting up a basic mod to work on for Project Zomboid.

WIP: this is cu...